

lux Connection-Scoped Store-Key Composition: a Z Specification

Formal model of DES-086's collision-impossibility invariant

August 2026

1 Introduction

Every scene or frame a client shows is installed under a store key. Before DES-086, that key was the literal string the client submitted — its `scene_id` — with no ownership check on the first write to a given id. Two different connections that happened to choose the identical string did not error; the second silently took over as owner, evicting the first's elements wholesale. This is the bug vox hit running in two Claude Code sessions at once: both pushed `scene_id="music-player"`, and the second overwrote the first on the shared display.

DES-086's fix composes the store key from the writing connection's own `ConnectionId` and the caller's local label, `ConnectionScopedId.compose(connection_id, local_id)`, at the one choke point every scene install passes through. A malformed `local_id` — blank, or carrying the reserved unit separator that joins the two halves — is rejected before composition ever runs, so a client cannot construct a `local_id` that would spell another connection's composed key.

Three properties must hold:

- I1 — collision-impossibility.** Two distinct connections never end up owning the same composed key. Equivalently: the composed key recorded for a connection's own write is always attributed to that connection, never to another one, across every reachable sequence of writes and reconnects.
- I2 — idempotency.** The same connection composing the same `local_id` twice always produces the same key, so a re-show updates the one scene it already owns rather than creating a second.
- I3 — reject before compose.** A `local_id` that carries the reserved separator is refused before it is ever composed into a key — the unrepresentable input is a boundary error, not a value the model ever has to reason about downstream.

The model abstracts the store key as the pair $(connection, local_id)$ rather than as a literal joined string. This is a faithful representation, not a simplification that begs the question: the separator's entire purpose is to make string concatenation behave like pairing — unambiguously decomposable back into its two halves — which is exactly what a Z pair already is by construction, *given* that `local_id` cannot itself carry the separator. That precondition is what I3 states and what the `taint` predicate below encodes. What the model-check verifies is not the trivial fact that distinct pairs are distinct — it is that no operation in the system, across any reachable interleaving, ever writes a connection's own key with someone else's value, and that no operation ever composes a key from a `local_id` the separator guard should have refused.

2 Basic Types

$[CONNECTION, LOCAL_ID]$

`CONNECTION` is the set of connections the Hub can distinguish — `ConnectionId` values, one per live MCP session or applet connection. `LOCAL_ID` is the set of caller-chosen labels a `show` call might submit, before any validation or composition. Two connections already exhibit the collision question (do connection c_1 's and c_2 's composed keys ever collapse); ProB's `DEFAULT_SETSIZE` bounds the carrier and a third connection confirms the invariant holds pairwise, not merely for one privileged pair.

3 Free Types

$TAINT ::= clean \mid tainted$

Whether a raw `local_id` carries the reserved unit separator — `ID_SEPARATOR` in the implementation, a control character no honest caller-chosen id ever contains. `clean`: composable. `tainted`: refused before composition, mirroring `ConnectionScopedId.__post_init__`’s guard.

4 Given Functions

$\mid \text{taint} : LOCAL_ID \rightarrow TAINT$

A total, given classification of every `local_id` in the carrier — the model does not construct actual character sequences (as the fuzz/ProB toolchain has no native string algebra convenient for this), so which ids are tainted is fixed axiomatically rather than derived. `DEFAULT_SETSIZE` guarantees ProB enumerates both a `clean` and a `tainted` instantiation of `LOCAL_ID` across its search, so the guard is genuinely exercised, not vacuously true.

5 State

$\frac{Store}{store : (CONNECTION \times LOCAL_ID) \mapsto CONNECTION}$
--

`store` is the Hub’s authoritative map from a composed key — the pair — to the connection that installed it. I1 and I3 are deliberately *absent* from this schema’s predicate: were either a state invariant, ProB would treat any operation that broke it as *disabled* rather than as a violation, and the fidelity check in the buggy sibling file could no longer reach the bad state it exists to reproduce. Both are instead stated as reachability goals in Section 8 and checked against the full reachable state space.

6 Initialization

$\frac{Init}{\frac{Store'}{store' = \emptyset}}$
--

No scene has been shown; the store holds no keys.

7 Operations

`Show` is a caller installing a scene: it composes its own connection with its own chosen `local_id` and writes the pair, attributed to itself. This is the whole of `SceneInstaller.install`’s composition step, modeled: no operation in this system ever writes a key attributed to a connection other than the one performing the write, because `Show` takes only its own `c?` as both the key’s first component and the value — there is no operation that could spell ”install under a key naming a connection other than the caller’s own.” Guarded on `clean`, so a tainted `local_id` is refused before it ever reaches the store (I3).

$\frac{Show}{\frac{\Delta Store}{\frac{c? : CONNECTION \quad l? : LOCAL_ID}{\frac{taint\ l? = clean}{store' = store \oplus \{(c?, l?) \mapsto c?\}}}}}$
--

`Reconnect` models a connection’s socket re-establishing — the DES-068 scenario named in the mission’s own framing, “composition against a `ConnectionId` mid-transition.” A reconnect does not change

which `ConnectionId` the session composes with (the identifier is stable for the connection's lifetime, derived once from the MCP transport's session key or the declared-identity hash); it leaves every already-composed key exactly as it was. Modeled as the identity transition on `store`, so the model-check exercises interleavings of `Show` and `Reconnect` freely and confirms a reconnect never disturbs an existing attribution.

<i>Reconnect</i>	_____
$\Delta Store$	
$c? : CONNECTION$	
$store' = store$	

8 Verification

I1 is checked as a reachability goal: the negating state

$$B_1 \hat{=} \exists c_1, c_2 : CONNECTION; l : LOCAL_ID \bullet c_1 \neq c_2 \wedge (c_1, l) \in \text{dom } store \wedge store(c_1, l) \neq c_1$$

— some entry keyed by c_1 's own composed pair ended up attributed to a different connection — must be unreachable. In ProB:

```
probccli docs/scene_id_namespacing.tex -model_check \
  -goal "#(c1,c2,l).(c1:CONNECTION & c2:CONNECTION & l:LOCAL_ID & \
    c1/=c2 & (c1|->l):dom(store) & store(c1|->l)/=c1)" \
  -p DEFAULT_SETSIZE 3 -p MAX_INITIALISATIONS 200
```

Reported never satisfied over the whole reachable state space: no sequence of `Show` and `Reconnect` calls, in any interleaving, ever attributes c_1 's own key to c_2 . This holds because the pair (c_1, l) and the pair (c_2, l) are distinct domain elements the instant $c_1 \neq c_2$, so `Show`'s override at (c_2, l) never touches (c_1, l) — the exact property that makes the vox collision unrepresentable rather than merely checked.

I3 is checked the identical way: the negating state

$$B_3 \hat{=} \exists c : CONNECTION; l : LOCAL_ID \bullet (c, l) \in \text{dom } store \wedge \text{taint } l = \text{tainted}$$

— a tainted `local_id` nonetheless reached the store — must be unreachable:

```
probccli docs/scene_id_namespacing.tex -model_check \
  -goal "#(c,l).(c:CONNECTION & l:LOCAL_ID & (c|->l):dom(store) & \
    taint(l)=tainted)" \
  -p DEFAULT_SETSIZE 3 -p MAX_INITIALISATIONS 200
```

Reported never satisfied: `Show`'s precondition $\text{taint } l? = \text{clean}$ disables the operation entirely for a tainted $l?$, so no tainted `local_id` ever composes into a key. The bare `-model_check` additionally confirms every operation is covered and the structural state predicate is preserved across the reachable space.

Deadlock. There is nothing here to deadlock: `Show` and `Reconnect` touch one piece of state under no lock at all in this model, because the design's own claim — confirmed, not merely repeated, by this exercise — is that composition is a pure, synchronous computation using data already resolved before the request begins, contending with nothing. Whenever a connection and a clean local id exist, `Show` is enabled, so the reachable state space has no stuck state; `Reconnect` is unconditionally enabled. `-cbc_deadlock` finds no state with every operation disabled.

9 Idempotency (I2)

I2 — the same connection composing the same `local_id` twice produces the same key — is a corollary of `Show`'s own definition rather than a claim with reachability content to search for. `Show` writes at the pair $(c?, l?)$; firing it twice with identical $c?$ and $l?$ computes the identical pair both times, so the second call re-writes the identical domain entry with the identical value $c?$ — a semantic no-op from

the store's perspective, which is exactly the "a re-show updates the one scene it already owns" behavior `HubDisplay.replace_scene` already gives a repeated `show()` on an unchanged `scene_id`. There is no interleaving or state-space question here: a mathematical function of two fixed arguments returns one fixed result on every call, by definition of being a function, and `ConnectionScopedId.compose` is exactly that — a total function of $(connection_id, local_id)$ with no hidden state. Unlike I1 and I3, forcing this through `-model_check` would search a state space with nothing left to find.